Scaling Laws and Compute

pig7selene

September 5, 2026

Abstract

Scaling laws summarize measured relationships among model size, training tokens, and compute; they are useful for planning only when their accounting conventions and empirical domain are explicit. This chapter distinguishes parameter count, corpus inventory, drawn-token budget, training FLOPs, and wall-clock time. It develops power-law fits and a simple compute-constrained allocation model, then contrasts the model-heavy Kaplan-style frontier with the more balanced Chinchilla-style result. The final sections describe data-limited and compute-limited regimes, the limits of extrapolation, and the accounting contracts required for an auditable pretraining plan.

1 Introduction

Chapters 5 through 8 describe what a decoder-only language model optimizes, which tokens reach that objective, how its parameters are updated, and how that computation remains numerically reliable. Scaling asks a different question: given a finite pretraining budget, how should a team allocate resources among model capacity, training tokens, and computation? The answer is not obtained by maximizing a single number such as parameter count. A larger model sees fewer tokens under a fixed compute budget; a longer run limits the model size that can be afforded. The relevant choice is a balance.

Empirical scaling laws make this balance measurable. They fit held-out loss across controlled runs at different scales and use the fitted relationship to estimate a useful frontier. Their value is practical: a small family of well-instrumented runs can rule out expensive configurations that are predictably data-starved or capacity-limited. Their status is equally important: they are regressions over a specified model family, dataset, objective, and optimization recipe. They are not fundamental laws of language or guarantees about downstream behavior.

This chapter uses the token-accounting distinctions of Chapter 6 and the valid-token loss of Chapter 5. It does not address how a fixed FLOP budget is distributed across devices; parallelism, communication, and cluster topology belong to Chapter 10.

2 Scale Accounting

2.1 Parameters, Documents, and Tokens

Let P denote a declared parameter count for the model. It is an architectural quantity: it counts stored trainable degrees of freedom under a stated convention. A report may count all trainable parameters, non-embedding parameters, or active parameters in a sparse model; these are not interchangeable. For the dense Transformer calculations below, P is the parameter count used by the FLOP convention.

Let R denote a raw corpus size, such as a document count, byte count, or source-record count. It is a property of an archive, not necessarily of training. Documents vary greatly in length; filtering, deduplication, and tokenization change their contribution; packing and loss masking can further change the number of supervised positions. Chapter 6 therefore distinguishes the unique token inventory N_{unique} from the drawn valid-token budget N_{drawn}. Here write

$$U = N_{\text{unique}}, \quad D = N_{\text{drawn}}, \tag{1}$$

where U is the inventory after a declared processing pipeline and D is the total valid target-token count actually consumed by the objective. The latter can exceed U when data are revisited or compo-

nents are upsampled. For a scaling study, D is usually the relevant data variable because it tracks the token-level training work. Raw document count R is not a substitute for D.

Quantity	Symbol and unit	What it measures
Parameter count	P parameters	Declared model capacity. The counting convention must specify, for example, whether embeddings or inactive sparse parameters are included.
Raw corpus size	R documents, bytes, or records	An archive property. It does not determine sequence length, valid targets, or model work.
Unique token inventory	U tokens	Retained tokenized content before repeated sampling, under a specified tokenizer and deduplication policy.
Effective token budget	D valid target tokens	The token positions actually drawn by the training objective, including mixture sampling and any reuse.
Training compute	C_{train} FLOPs	Arithmetic work implied by the model, token budget, and FLOP-accounting convention.
Wall-clock time	t_{wall} seconds	Elapsed execution time, which also depends on sustained throughput and operational interruptions.

Table 1: Scale variables have different units and different roles. A credible plan reports each one rather than treating raw corpus size, token budget, FLOPs, and elapsed time as synonyms.

The ratio $\frac{D}{P}$, often called tokens per parameter, is a useful diagnostic only after both numerator and denominator have been defined. It compares training exposure to model capacity under a particular accounting convention; it does not measure data quality, coverage, or the number of optimization updates. It is also not fixed by the mathematics of language modeling. It emerges from a fitted frontier and may change when the data mixture or model family changes.

2.2 Training FLOPs and Wall-Clock Time

For a dense decoder-only Transformer, a common planning approximation expresses pretraining arithmetic as

$$C_{\text{train}} \approx kPD, \quad k \approx 6. \tag{2}$$

The factor near six summarizes an approximate forward-and-backward cost per parameter-token pair under a conventional dense-model counting rule. It is useful because it exposes the dominant first-order trade-off: at fixed C_{train}, increasing P requires reducing D, and conversely. It is not a precise bill of materials. Attention work, embeddings, sequence length, activation recomputation, sparsity, kernel choices, and the definition of a FLOP change the constant and sometimes the form of the estimate. Kaplan et al. use a related parameter-and-token accounting to study compute-efficient language-model training [1].

FLOPs are a work estimate, whereas wall-clock time is an execution outcome. If r_{eff} is the measured sustained training rate in FLOPs per second for a fixed implementation, then a planning estimate is

$$t_{\text{wall}} \approx \frac{C_{\text{train}}}{r_{\text{eff}}}. \tag{3}$$

Peak accelerator throughput is not r_{eff}. Input stalls, non-matmul work, memory traffic, numerical precision, and recovery time lower sustained performance. This chapter needs only the distinction: two runs with the same declared FLOPs can take different elapsed time, and two runs with the same

elapsed time can execute different FLOPs. The mechanisms behind sustained throughput are a systems topic, not a replacement for FLOP accounting.

3 Empirical Power-Law Behavior

A common one-variable scaling fit is

$$\mathcal{L}(x) = Ax^{-\alpha} + B, \quad A > 0, \quad \alpha > 0. \tag{4}$$

Here $\mathcal{L}$ is a measured validation loss and x can stand for a scale variable such as P, D, or an optimally allocated compute budget. The coefficient A sets the scale of the reducible loss, α controls the rate of improvement, and B is a fitted asymptote or floor. Within this equation, B is unrelated to the batch-size notation used in Chapter 7. To avoid that collision below, write the asymptote as $\mathcal{L}_{\text{inf}}$.

The practical content of Equation 4 is diminishing return rather than a promise of unlimited improvement. If x is multiplied by r, then the excess loss $\mathcal{L}(x) - B$ changes by $r^{-\alpha}$. A positive exponent therefore predicts consistent relative improvement on a log-log scale, but progressively smaller absolute gains as the reducible term shrinks. After subtracting a justified fitted floor, the relationship is linear in logarithms:

$$\log(\mathcal{L}(x) - B) = \log A - \alpha \log x. \tag{5}$$

Kaplan et al. reported smooth power-law relationships between autoregressive validation loss and model size, dataset size, and compute in their experimental setting [1]. Such a fit should be judged by held-out residuals, by its stability across nearby scales, and by whether its accounting conventions match the proposed run. A straight segment on one log-log plot does not justify extrapolation across a new tokenizer, data distribution, architecture, or training procedure.

4 Compute-Constrained Allocation

4.1 A Representative Joint Loss Surface

A one-variable fit hides the fact that a model can be limited by either capacity or training exposure. A useful illustrative surface is

$$\hat{\mathcal{L}}(P, D) = \mathcal{L}_{\text{inf}} + a_P P^{-\alpha} + a_D D^{-\beta}, \quad a_P, a_D, \alpha, \beta > 0. \tag{6}$$

The P term represents the capacity-limited contribution and the D term the token-limited contribution. This separable form is a planning model, not the only valid scaling-law parameterization. Real fits can include interaction terms, finite-data corrections, learning-rate-schedule effects, or a different functional form. Its advantage is that it makes the allocation logic explicit.

Suppose the planned runs obey the approximate compute relation Equation 2, so that $D = \frac{C_{\text{train}}}{kP}$. Substituting this constraint into Equation 6 gives a loss along one compute frontier:

$$\hat{\mathcal{L}}(P; C_{\text{train}}) = \mathcal{L}_{\text{inf}} + a_P P^{-\alpha} + a_D \left(k \frac{P}{C_{\text{train}}} \right)^{\beta}. \tag{7}$$

The first term decreases with model size; the second increases because a larger model consumes the same FLOPs over fewer tokens. Setting the derivative with respect to P to zero equates their marginal contributions:

$$\alpha a_P P^{-\alpha} = \beta a_D D^{-\beta}. \tag{8}$$

This yields the qualitative compute-optimal exponents

$$P^* = c_P C_{\text{train}}^{\frac{\beta}{\alpha+\beta}}, \quad D^* = c_D C_{\text{train}}^{\frac{\alpha}{\alpha+\beta}}, \quad c_P, c_D > 0. \tag{9}$$

The fitted exponents determine where additional compute goes. If α and β are similar, model size and token budget grow at similar rates. If they differ, the balance shifts. The derivation does not establish the values of α and β; it shows why different empirical fits can recommend different allocations under the same high-level FLOP constraint.

4.2 Kaplan-Style and Chinchilla-Style Frontiers

Table 2 contrasts two influential results. The difference is historical and empirical, not a contradiction in algebra. The two studies fit different data and modeling assumptions, then minimized their respective fitted loss models subject to a compute budget.

Study	Compute-optimal implication	Planning consequence
Kaplan et al. (2020)	The reported compute-efficient frontier grew parameters approximately as $C^{0.73}$ and data more slowly, approximately as $C^{0.27}$.	A relatively model-heavy allocation, with models stopped well before convergence on a much larger token inventory.
Hoffmann et al. (2022)	The controlled fit found that model size and training tokens should grow in roughly equal proportion as compute increases.	Allocate materially more tokens per parameter; the published Chinchilla run used 70B parameters and 1.4T tokens.

Table 2: Two influential compute frontiers. The exponents and ratios summarize specific empirical fits; neither row supplies a tokenizer-, dataset-, or architecture-independent prescription.

Kaplan et al. found a model-heavy allocation in which the fitted parameter scale grew approximately as $C_{\text{train}}^{0.73}$ and the required data scale approximately as $C_{\text{train}}^{0.27}$ [1]. In this regime, a very large model could be compute-efficient even though it was trained far short of convergence on the available corpus. This recommendation was consequential because it separated compute-efficient training from the intuition that every model should be trained until its loss no longer improves.

Hoffmann et al. revisited the allocation with more than four hundred Transformer runs spanning model sizes and token budgets, and found a substantially more balanced frontier: doubling model size should be accompanied by a doubling of training tokens for compute-optimal training [2]. Their compute-matched Chinchilla model used 70 billion parameters and 1.4 trillion training tokens, about 20 tokens per parameter, while using the same pretraining compute budget as the 280-billion-parameter Gopher model. It outperformed Gopher on their reported evaluations [2]. This result changed the default planning question from "how large can the model be?" to "which model-and-token pair uses the budget most effectively?"

5 Regimes, Reuse, and Bottlenecks

5.1 Undertraining and Overtraining

A model is **undertrained** relative to a chosen scaling frontier when it has too little token exposure for its parameter count and compute budget. Its loss can still be decreasing rapidly when training stops, and a smaller model trained on more data may achieve better validation loss for the same C_{train}. Undertraining is therefore not a claim that the model has seen few documents; it is a relationship among P, D, the data distribution, and the training recipe.

The opposite label, **overtraining**, must be used carefully. It can describe a run that spends many additional tokens for small marginal validation improvement, or one that repeatedly reuses a limited corpus until generalization deteriorates. These are different mechanisms. A large supply of new, high-quality, diverse tokens can continue to improve a fixed model beyond a compute-optimal point chosen for pretraining efficiency. By contrast, repeated exposure to duplicates or narrow domains can inflate D without comparable new information. As Chapter 6 emphasizes, mixture weights, deduplication, and provenance determine what a drawn-token count means.

No universal tokens-per-parameter threshold separates these regimes. The Chinchilla ratio is a result of a particular controlled study, not a license to treat every token as interchangeable or every parameter count as comparable. Evaluation goals also matter: a model selected for low pretraining loss under a one-time compute budget can differ from a model selected for low inference cost across a long deployment horizon.

5.2 Data-Limited and Compute-Limited Planning

In a compute-limited regime, the team has enough validated data to consider several (P, D) pairs satisfying a chosen compute budget. The principal task is to estimate a frontier from compatible pilot runs and select a pair near its minimum. The learning-rate schedule, batch convention, and sequence construction must stay comparable across those pilots; otherwise the fitted difference mixes scale with recipe changes.

In a data-limited regime, the supply of high-quality, policy-compliant, sufficiently diverse tokens constrains D before the compute budget is exhausted. Replaying the same inventory changes the data-reuse rate rather than creating a larger corpus. The planning problem then includes a data decision: accept reuse with an explicit risk assessment, improve curation and collection, reduce model capacity, or spend compute elsewhere. Calling the run "Chinchilla-optimal" without stating the available unique inventory conceals this constraint.

6 Scaling Predictions and Their Limits

Scaling laws are most defensible as interpolation tools. A team can train a family of smaller models using the intended tokenizer, data mixture, sequence length, loss reduction, optimizer schedule, and evaluation set; fit a loss surface; and compare its predictions with additional held-out scales. The validation loss being predicted must be measured on a fixed, protected evaluation distribution. A changing benchmark or contaminated validation set can create a persuasive but meaningless curve.

Extrapolation is harder. Changes in architecture, active-parameter sparsity, context length, data quality, curriculum, precision policy, optimization stability, or evaluation distribution can alter both the fitted constants and exponents. A loss floor may be unidentifiable from a limited range, so subtracting a guessed B in Equation 5 can make a log-log fit appear straighter than its evidence warrants. More fundamentally, cross-entropy loss is only one outcome: downstream reliability, calibration, safety, and task-specific capability need not follow the same scaling relationship.

The correct conclusion is neither that scaling laws are exact nor that they are useless. They are empirical summaries that can convert a large, uncertain budget into testable predictions. Their assumptions, residuals, data identity, and out-of-sample checks are part of the result.

7 Practical Planning Under a Fixed Budget

Begin with four independently measured constraints: a maximum training FLOP budget, a wall-clock deadline, a verified unique-token inventory, and an intended evaluation target. Convert each candidate architecture into a declared P, estimate feasible token budgets D with Equation 2, and reject pairs that exceed the data, compute, or time constraints. The result is a short set of feasible configurations, not yet an optimal choice.

Next, run scaled pilots with the same data-processing policy and optimization semantics that the full run will use. Record validation loss against both D and C_{train}, not merely against update count. A pilot that changes model size while also changing tokenizer, sequence length, mixture, warmup clock, or loss-mask reduction cannot identify a model-size effect cleanly. Chapter 7′s schedule and batch conventions remain part of the experimental definition even though this chapter does not rederive them.

Finally, choose a conservative point near the fitted frontier rather than a single extrapolated optimum. Preserve a reserve for failed runs, checkpoint recovery, ablations, and validation. If inference cost is material, include it as a separate deployment constraint: a smaller model trained longer may be preferable even when a pretraining-only frontier would select a larger one. The plan should state which objective it optimizes, because "best model" has no meaning without a budget and a use case.

8 Implementation Contracts

A scaling report must make its counters auditable. It should record the parameter-count convention, including whether embeddings, tied heads, inactive experts, or frozen tensors are included. It should record raw corpus size R, unique inventory U, and drawn valid-token budget D separately, together with tokenizer version, data manifest, mixture schedule, sequence-construction policy, and loss-mask convention. A token counter that includes padding or ignored labels is not a counter for the objective in Chapter 5.

The compute contract must name the FLOP formula, its constant k, and any excluded work. It should distinguish estimated compute from measured elapsed time and record the measured sustained rate used in Equation 3. Checkpoints and experiment logs should preserve the update count, valid-token count, data-reuse rate, learning-rate state, model configuration, and evaluation-dataset version. These fields make it possible to reproduce a scaling point and to detect when two nominally equal-compute runs are not actually comparable.

Before relying on an extrapolation, retain pilot configurations that test both model-limited and token-limited sides of the proposed optimum, hold out at least one scale for validation, and compare prediction error in the loss domain rather than only in a visually appealing log-log plot. A scaling fit is an implementation artifact as well as a mathematical fit: its conclusions are only as reliable as the counters and controlled variables that produced it.

9 Summary

Pretraining scale has several independent units. Parameter count measures declared model capacity; raw document count describes an archive; unique and drawn tokens describe different forms of data exposure; FLOPs estimate arithmetic work; and wall-clock time measures execution. Token count is usually the meaningful scale variable for the pretraining objective because it records valid target positions actually consumed, whereas document count does not.

Empirical power laws describe diminishing improvements over a measured regime, and a joint loss model explains why a fixed compute budget creates a trade-off between larger models and more training tokens. Kaplan-style fits favored a more model-heavy allocation, while Chinchilla-style results favored substantially more data relative to parameters. Neither recommendation is universal. Data quality, reuse, model family, objective, and deployment requirements all change the decision. A sound pretraining plan therefore uses scaling laws as validated, fully accounted predictions rather than as a single immutable ratio.

References

[1] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020, doi: 10.48550/arXiv.2001.08361.

[2] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022, doi: 10.48550/arXiv.2203.15556.